

Secure Elder-Care RAG Assistant

A beginner-friendly 10-week learning + building roadmap

The rule of this roadmap: do not try to learn the whole AI architecture first. Each week gives you one piece of the system. First learn only what you need for that week's milestone, then implement it. By Week 10, the pieces become one complete portfolio project.

Project in one sentence

Build a fake elder-care management system where authorized caregivers can ask natural-language questions about residents, while the backend enforces which residents each user is allowed to access. Use synthetic data only — never real patient/health data.

Target stack

Layer	Beginner choice
Backend/API	Python + FastAPI
Main database	PostgreSQL
Vector database	Qdrant (local Docker for the project)
Embeddings	A sentence-transformer / Hugging Face embedding model
LLM	Start with a normal API for learning; later experiment with a local/open model
Authentication	FastAPI OAuth2/JWT concepts
Containers	Docker + Docker Compose
Frontend	Very simple HTML/JS or Streamlit — keep it secondary

The final architecture you are working toward

USER → Frontend → FastAPI → Authentication/Authorization → Permission-aware Retrieval → PostgreSQL + Qdrant → LLM → Answer + Audit Log

Important security principle: the LLM does not decide what a user is allowed to see. Your backend determines the user's access scope first, then retrieval is restricted to that scope.

10-week roadmap

Week 1 — Understand the system + design the data model

Why now: Before coding AI, understand what the application stores.

Learn:

- Learn: relational databases, tables, rows, primary keys, foreign keys, relationships, basic SQL.
- Learn just enough PostgreSQL to create/query tables and join related records.
- Design: users, roles, residents, care_reports, care_goals, incidents, and user_resident_permissions.
- Create synthetic residents and care reports. No real health data.

Build / exit condition: A PostgreSQL database containing fake residents, users, roles, permissions, care reports and incidents. You should be able to answer normal SQL questions such as: 'show reports for resident 3 from the last 7 days.'

Resources:

- [PostgreSQL Tutorial](#)
- [FastAPI SQL databases](#)

Week 2 — Build the backend API

Why now: Turn the database into an application instead of manually querying SQL.

Learn:

- Learn: HTTP, REST, JSON, GET/POST, path parameters, request bodies, status codes.
- Learn basic FastAPI routing, validation and database integration.
- Implement endpoints for residents, reports, goals and incidents.
- Keep the API simple; do not add an LLM yet.

Build / exit condition: You can use an API client/browser docs to create and retrieve fake resident information through FastAPI.

Resources:

- [FastAPI Learn](#)
- [FastAPI SQL databases](#)

Week 3 — Authentication + RBAC / permissions

Why now: Make the application understand who the user is and what they are allowed to access.

Learn:

- Learn: authentication vs authorization.
- Learn: password hashing, JWT/Bearer tokens, roles and permission checks.
- Implement users: admin and caregiver.
- Implement a permission rule such as: admin → all residents; caregiver A → residents 1 and 2; caregiver B → resident 3.
- Every resident-data endpoint must enforce authorization on the server.

Build / exit condition: A caregiver cannot retrieve a resident they are not assigned to, even if they manually call the API or change the resident ID.

Resources:

- [FastAPI Security](#)
- [FastAPI OAuth2 + JWT](#)

Week 4 — Embeddings + Vector Database

Why now: Now add the semantic-search part of the system.

Learn:

- Learn: embeddings, vectors, similarity search, cosine similarity, metadata/payload.
- Learn why the vector DB is not the source of truth; PostgreSQL remains your authoritative application database.
- Run Qdrant locally.
- Chunk care reports and create embeddings.
- Store vectors with metadata such as resident_id, report_id, date and document_type.
- Test semantic search without an LLM.

Build / exit condition: Given a question such as 'Has this resident fallen recently?', Qdrant returns relevant fake reports. Also prove that filtering by resident_id works.

Resources:

- [Qdrant Documentation](#)
- [Qdrant Filtering](#)
- [Qdrant beginner payload filtering](#)

Week 5 — Build basic RAG

Why now: Connect retrieval to an LLM and produce grounded answers.

Learn:

- Learn: RAG, chunking, retrieval, context, prompt construction and grounded generation.
- Implement: question → embedding → Qdrant search → retrieved reports → prompt → LLM → answer.
- Tell the model to answer only from retrieved evidence.
- Return the source reports/citations used for the answer.

Build / exit condition: The chatbot can answer questions about your synthetic residents using retrieved reports instead of relying only on model memory.

Resources:

- [DeepLearning.AI RAG Course](#)
- [Qdrant Documentation](#)

Week 6 — Permission-aware RAG

Why now: This is the most important week for the job you showed me.

Learn:

- Combine Week 3 authorization with Week 5 retrieval.
- Before retrieval, determine which resident IDs the authenticated user can access.
- Pass that access scope into the vector search filter.
- Do not rely on the LLM prompt to enforce permissions.
- Test attacks: ask for another resident, change IDs, ask indirect questions, and attempt to retrieve forbidden documents.

Build / exit condition: Caregiver A can ask anything about residents 1 and 2 but receives no information about resident 3. Admin can access all residents.

Resources:

- [Qdrant Filtering](#)
- [FastAPI Security](#)

Week 7 — Tool calling / controlled actions

Why now: Move from 'one giant RAG search' toward a more realistic assistant.

Learn:

- Learn what tool/function calling is.
- Create controlled backend tools such as `get_recent_reports()`, `get_fall_incidents()`, `get_care_goals()`.
- The LLM may request a tool, but the backend validates the authenticated user's permissions before executing it.
- Keep tools read-only for this project.

Build / exit condition: Questions like 'Has Mrs Smith fallen recently?' can trigger the correct backend function, while unauthorized tool calls are rejected.

Resources:

- [FastAPI documentation](#)
- [OWASP GenAI Security Project](#)

Week 8 — Security, hallucination mitigation + evaluation

Why now: Make the demo behave like an engineered system rather than a chatbot toy.

Learn:

- Learn: prompt injection, insecure output handling, sensitive information disclosure, excessive agency.
- Create a small evaluation dataset: question, expected answer, expected source documents, allowed user.
- Measure retrieval quality and whether answers stay grounded in sources.
- Add 'I don't have enough information' behavior.
- Never put fake/real patient data into debug logs unnecessarily.
- Add basic audit events: user, timestamp, question, resident scope, retrieved document IDs, response status.

Build / exit condition: You have repeatable tests showing: correct retrieval, permission isolation, grounded answers and safe refusal when evidence is missing.

Resources:

- [OWASP LLM/GenAI Top 10](#)
- [OWASP current LLM Top 10 page](#)
- [DeepLearning.AI RAG Course](#)

Week 9 — Docker + deployment architecture + data residency

Why now: Learn how the pieces run together and what 'architecture' means in production.

Learn:

- Learn Docker images, containers, volumes, networks and Docker Compose.
- Containerize FastAPI and Qdrant; run PostgreSQL as a service/container for the demo.
- Draw your architecture diagram.
- Understand public vs private services, TLS, secrets, backups, encryption, IAM and region selection.
- For the Swiss requirement, research a deployment where sensitive data and model inference stay inside the permitted Swiss environment. For the portfolio project, clearly label this as a simulated architecture unless you actually deploy in Switzerland.

Build / exit condition: One command can start your local stack. Your README contains a clear architecture diagram and explains where sensitive data would flow.

Resources:

- [Docker Get Started](#)
- [Designing Machine Learning Systems — Chip Huyen](#)

Week 10 — Integration, portfolio polish + final architecture review

Why now: Turn the project into something you can show a client.

Learn:

- Add a simple chat UI.
- Show logged-in user and accessible residents.
- Show citations/source reports in answers.
- Add tests for RBAC, retrieval filtering, prompt injection and hallucination cases.
- Write a README: problem → requirements → architecture → security → data flow → limitations → how to run.
- Create a final architecture diagram.
- Record a 2–4 minute demo.

Build / exit condition: A complete portfolio project demonstrating secure, permission-aware RAG over private synthetic enterprise data.

Resources:

- [Hugging Face LLM Course](#)
- [Designing Machine Learning Systems — Chip Huyen](#)

Resource library — use these instead of randomly searching

Core resources

PostgreSQL official tutorial

Best for Week 1. Covers databases, tables, queries, joins, foreign keys and transactions.

<https://www.postgresql.org/docs/current/tutorial.html>

FastAPI Learn + Security

Best for Weeks 2–3. Use the tutorial to build the API, then learn OAuth2/JWT and authorization concepts.

<https://fastapi.tiangolo.com/learn/>

Qdrant Documentation

Best for Week 4 and Week 6. Especially study payload metadata and filtering.

<https://qdrant.tech/documentation/>

DeepLearning.AI — Retrieval Augmented Generation

Best for Weeks 5–6 and part of Week 8. It covers retrieval, vector databases, chunking, hybrid search and evaluation.

<https://www.deeplearning.ai/courses/retrieval-augmented-generation>

Hugging Face LLM Course

Free. Use it selectively for LLM/Transformer/embedding fundamentals. Do not try to finish the entire course before starting the project.

<https://huggingface.co/learn/llm-course/en/chapter1/1>

OWASP GenAI Security Project

Best for Week 8 onward. Learn prompt injection, sensitive information disclosure and other LLM application risks.

<https://genai.owasp.org/>

Docker Get Started

Best for Week 9. Learn containers, images, volumes and basic application stacks.

<https://docs.docker.com/get-started/>

Book — use later, not as a prerequisite

Designing Machine Learning Systems — Chip Huyen. This is excellent for learning the system-design mindset: requirements, reliability, deployment, infrastructure, monitoring and production trade-offs. It is intermediate/advanced, so read selected chapters after you have built the first version rather than trying to finish it first.

[O'Reilly book page](#)

What NOT to learn yet

Do not let the job post convince you that you need Kubernetes, fine-tuning, multi-agent systems, advanced cloud networking, distributed systems, MLOps platforms or complex frontend engineering before starting. Those are later topics. The portfolio project is specifically designed to teach you the architecture by assembling small pieces.

Weekly working rhythm

Suggested pace: 8–12 hours/week.

1) 30–60 min: understand the week's concepts.

2) 2–4 hours: follow one focused tutorial/resource.

3) 4–7 hours: implement the week's milestone yourself.

4) 30 min: write what you learned in README/notes.

If a week takes two weeks, that is completely fine. The order matters more than the calendar.

Definition of done for the final portfolio

- Synthetic elder-care data only.
- PostgreSQL is the source of truth for application data.
- Qdrant stores embeddings plus metadata for retrieval.
- Authentication identifies the user.
- Backend authorization determines which residents the user can access.
- Vector retrieval is filtered by the user's allowed resident scope.
- LLM never independently decides authorization.
- Answers are grounded in retrieved evidence and show source reports.
- The assistant can refuse when evidence is missing.
- Tool calls are validated server-side.
- Audit events are recorded without unnecessarily storing sensitive content.
- Docker can reproduce the local system.
- Architecture diagram and security/data-flow explanation are included.
- README clearly distinguishes the portfolio simulation from a real Swiss healthcare deployment.

Most important mindset: You are not trying to become an expert in every technology named in the job post. You are building one system and learning each component exactly when the project needs it. That is the reason this roadmap starts with the database and only reaches RAG, security and cloud architecture later.